УНИВЕРСИТЕТ ИТМО

Системы искусственного интеллекта

ОТЧЕТ ПО МОДУЛЮ 1

Базы знаний и онтологии

Лабораторная работа 1: Prolog и OWL
Лабораторная работа 2: система поддержки принятия решений

Предметная область: производственные цепочки игры Factorio

Санкт-Петербург · 2026

2. Аннотация

В модуле разработана учебная база знаний о производственных цепочках Factorio на языке Prolog. Она описывает сырье, промежуточные продукты, рецепты, машины и доступность машин на двух планетах. На основе фактов построена OWL-онтология в формате Turtle, открываемая в Protégé. Для лабораторной работы 2 создана командная система поддержки принятия решений, которая после короткого диалога выбирает машину для заданного продукта. Рекомендация формируется запросом к Prolog и сопровождается объяснением критерия выбора. Корректность основных запросов, структурной модели OWL и сценариев DSS подтверждена автоматическими тестами; запуск внешнего OWL reasoner требует Protégé.

3. Введение

Цель модуля — освоить логическое моделирование, построение онтологий, проверяемый вывод и применение знаний в рекомендательной системе. В производственном планировании игры вопрос «чем изготовить продукт?» зависит сразу от рецепта, класса машины, планеты, энергии и выбранного критерия. Явные факты и правила позволяют повторно использовать эти знания, проследить основание ответа и проверить результат отдельными запросами.

Граница модели намеренно узкая: 10 продуктов, 7 машин и две планеты. Модель служит демонстрацией метода; она не претендует на полный справочник всех рецептов и обновлений игры. Доступность означает местное получение машины без межпланетной перевозки. Все результаты ниже получены из файлов проекта в версии на дату отчета.

Артефакты

Артефакт	Роль
src/lab1/factorio.pl и base/*.pl	Факты, рецепты и точка загрузки Prolog
src/lab1/rules.pl	Логический вывод
src/lab1/ontology/factorio.ttl	OWL-онтология для Protégé
src/lab2/dss.py	Диалог и ранжирование рекомендаций
tests/	Проверка Prolog, OWL-структуры и DSS

4. База знаний в Prolog: дизайн и реализация

4.1. Словарь и структура

Одноместные факты обозначают 22 вида сырья, 10 промежуточных продуктов и 7 машин — всего 39 объектов. Двухместные отношения machine_type/2, power_source/2, machine_speed/2, machine_power/2, available_on/2 и recipe_type/2 связывают объекты со свойствами; одних только machine_type/2 и power_source/2 имеется 14 фактов. Трехместные ingredient/3 задают 14 пар «продукт — материал — количество».

Предикат	Смысл	Пример
resource/1	Сырьевой ресурс	resource(iron_ore)
intermediate_product/1	Изготавливаемый продукт	intermediate_product(steel_plate)
machine_type/2	Категория устройства	machine_type(steel_furnace,furnace)
ingredient/3	Материал и количество	ingredient(steel_plate,iron_plate,5)
recipe_type/2	Категория для рецепта	recipe_type(iron_plate,furnace)
available_on/2	Планета работы	available_on(foundry,vulcanus)

Исходный проект содержал неполную загрузку файлов, разнобой между craft_from и crafted_from, неверные аргументы и ссылки на отсутствующие продукты. Факты сведены к одному словарю; составной рецепт электронной схемы теперь проходит через медный кабель. Каждый представленный продукт имеет выход, ингредиенты и тип машины.

4.2. Правила

Семь правил выводят твердые ресурсы, доступность производства, фильтр энергии, транзитивное сырье, использование жидкости, альтернативы машин и сравнение скорости. Правило solid_resource/1 использует отрицание fluid_resource/1. raw_material/2 имеет базовый и рекурсивный случаи, поэтому электронная схема приводит к iron_ore и copper_ore через промежуточные продукты.

```
can_craft(M,P,L) :- machine_type(M,T),
    recipe_type(P,T), available_on(M,L).
raw_material(P,R) :- ingredient(P,R,_), resource(R).
raw_material(P,R) :- ingredient(P,C,_),
    intermediate_product(C), raw_material(C,R).
```

4. База знаний в Prolog: запросы

Ниже приведены ответы SWI-Prolog 10.0.2. Запросы охватывают прямые факты, переменные, конъюнкцию, дизъюнкцию, отрицание и срабатывание правил.

№ / запрос	Фактический ответ
1. resource(iron_ore).	true
2. machine_type(steel_furnace,T).	T = furnace
3. findall(M,can_craft(M,iron_plate,nauvis),Ms).	Ms = [stone_furnace,steel_furnace,electric_furnace]
4. solid_resource(water).	false
5. powered_crafter(electric_furnace,iron_plate,nauvis,electricity).	true
6. setof(R,raw_material(electronic_circuit,R),Rs).	Rs = [copper_ore,iron_ore]
7. uses_fluid(sulfur).	true
8. faster_than(steel_furnace,stone_furnace).	true
9. can_craft(foundry,molten_iron,nauvis).	false
10. can_craft(foundry,molten_iron,vulcanus); can_craft(chemical_plant,molten_iron,vulcanus).	true

Запрос 3 объединяет тип рецепта и доступность на Наувисе; запрос 5 добавляет источник энергии; запрос 6 делает многошаговый вывод по цепочке ингредиентов. Отрицательный ответ 9 следует из отсутствия факта available_on(foundry,nauvis). В Prolog применяется допущение о замкнутом мире: непредставленный факт считается недоказанным.

5. Онтология в Protégé: моделирование

Онтология создается скриптом build_ontology.py непосредственно из тех же фактов Prolog. Выход — Turtle-файл factorio.ttl с пространством имен https://example.org/factorio#. Его можно открыть в Protégé командой File → Open. В нем представлены 10 рецептов, 14 объектов IngredientUse, ресурсы, продукты, машины, планеты и источники энергии.

5.1. Классы и ограничения

• Иерархия Material → IntermediateProduct → MetalPlate имеет три уровня; ProductionEntity → Machine → Furnace/Assembler/ChemicalPlant/Foundry задает иерархию оборудования.

• RawResource и IntermediateProduct несовместимы; SolidResource и FluidResource несовместимы. Для отдельных типов машин заданы owl:disjointWith.

• Machine ограничен ровно одним poweredBy из класса PowerSource; Recipe ограничен ровно одним produces из класса IntermediateProduct; IngredientUse — ровно одним material.

• quantity и yield имеют тип xsd:positiveInteger, powerKW — xsd:nonNegativeInteger, speed — xsd:decimal.

5.2. Перевод Prolog → OWL

Prolog	OWL	Пояснение
machine(stone_furnace)	stone_furnace a Furnace	Тип индивида
machine_type(M,furnace)	Furnace ⊑ Machine	Категория как класс
available_on(M,nauvis)	M availableOn nauvis	ObjectProperty
power_source(M,fuel)	M poweredBy fuel	ObjectProperty
machine_speed(M,2.0)	M speed 2.0^^xsd:decimal	DataProperty
ingredient(P,R,Q)	Recipe → IngredientUse → R; quantity Q	Трехместное отношение как отдельный объект

Генератор проверяет, что каждый ингредиент ссылается на известный ресурс или продукт, а все машины имеют тип, источник энергии, скорость и мощность. Правила can_craft/3 и raw_material/2 не переводятся в OWL автоматически: OWL хранит исходные связи, а вывод Prolog выполняется в DSS.

Источники предметных данных

Рецепты и характеристики сверены по Factorio Wiki: Electronic circuit, Steel furnace, Assembling machine 1, Foundry. Критерии задания и форма отчета взяты из двух предоставленных PDF.

5. Онтология: запросы и проверки

5.3. Компетентностные DL-запросы

Следующие запросы предназначены для вкладки DL Query после запуска HermiT в Protégé. Указаны ожидаемые ответы по материализованным фактам онтологии; фактический запуск HermiT/ELK в этой среде не выполнялся.

DL Query	Ожидаемые индивиды
MetalPlate	iron_plate, copper_plate, steel_plate
Furnace and availableOn value nauvis	stone_furnace, steel_furnace, electric_furnace
Machine and poweredBy value fuel	stone_furnace, steel_furnace
Recipe and produces value molten_iron	molten_iron_recipe
IngredientUse and material value copper_cable	ingredient_use_8
Machine and supportsRecipe value sulfur_recipe	chemical_plant

5.4. Подтвержденные структурные проверки

SWI-Prolog загрузил Turtle как RDF и проверил связь рецепта с продуктом, локальную доступность литейной только на Вулканусе, цепочку подклассов MetalPlate → IntermediateProduct → Material → ProductionEntity, ингредиенты электронной схемы, ограничение кардинальности poweredBy и отсутствие индивидов, явно отнесенных одновременно к несовместимым классам. Эти тесты проверяют структуру и некоторые логические предпосылки, но не заменяют полную проверку консистентности OWL reasoner.

6. DSS: архитектура и реализация

Программа src/lab2/dss.py принимает одну строку фиксированного вида:

```
Продукт: железная плита; Планета: Наувис;
Приоритет: скорость; Энергия: электричество
```

Поля «Продукт», «Планета» и «Приоритет» обязательны; отсутствующие значения система спрашивает отдельно. «Энергия» необязательна и по умолчанию равна «любая». Значения нормализуются через закрытый словарь русских имен и переводятся в атомы Prolog; произвольный текст в запрос не подставляется. Неизвестное поле, значение или повтор поля вызывает понятную ошибку.

6.1. Конвейер

Шаг	Реализация
1. Разбор	Разделение по «;» и «:», проверка фиксированных слотов
2. Уточнение	Диалог о недостающем продукте, планете и приоритете
3. Логический запрос	can_craft(M,P,L), power_source(M,E), machine_speed(M,S), machine_power(M,W)
4. Ранжирование	Скорость: больше S, затем меньше W; электроэнергия: меньше W, затем больше S
5. Ответ	Имя машины, объяснение по фактам и альтернативы

6.2. Формирование Prolog-запроса

```
findall(_{machine:M,speed:S,power_kw:W,power_source:E},
  (can_craft(M,iron_plate,nauvis),
   power_source(M,electricity),
   machine_speed(M,S), machine_power(M,W),
   power_source(M,E)), R).
```

Запрос выполняется отдельным процессом SWI-Prolog, результат передается как JSON. Приоритет применяется к списку кандидатов после логической фильтрации. Нулевое значение powerKW у топливных машин означает отсутствие электрического потребления, а не нулевые общие затраты.

6. DSS: сценарии и обработка исключений

Ниже показаны запуски CLI с заданной строкой; запрос во всех случаях основан на can_craft/3, при ограничении энергии дополнительно на power_source/2. В таблице указаны решающие условия и фактический ответ.

Вход: продукт / планета / приоритет / энергия	Результат и основание
Железная плита / Наувис / скорость / электричество	Электрическая печь: furnace, Наувис, electricity, скорость 2.0.
Железная плита / Наувис / скорость / топливо	Стальная печь: скорость 2.0 против 1.0 у каменной.
Железная шестерня / Наувис / электроэнергия / электричество	Сборочный автомат 1: 75 кВт против 150 кВт.
Расплавленное железо / Вулканус / скорость / любая	Литейная: найден единственный совместимый тип, скорость 4.0.
Электронная схема / Наувис / скорость / любая	Сборочный автомат 2: скорость 0.75 против 0.5.
Расплавленное железо / Наувис / скорость / любая	Нет подходящей машины; предложено изменить условия или планету.

Нештатные случаи

• Неизвестный продукт («Продукт: неизвестный») отвергается с перечнем допустимых вариантов.

• Повтор «Планета: Наувис; Планета: Вулканус» распознается как конфликт ввода, даже если значения одинаковы.

• Пустой результат не подменяется случайной рекомендацией.

• Если введен только продукт, система задает два вопроса о планете и приоритете; завершение ввода до ответа дает ошибку.

7. Тестирование и отладка

Набор	Объем	Подтверждено
Prolog / tests/test_lab1.pl	13 тестов	Факты, переменные, и/или/не, рекурсия, планеты, целостность рецептов
OWL / tests/test_ontology.pl	6 тестов	RDF-загрузка, связи, иерархия, ограничения, явная несовместимость
DSS / tests/test_dss.py	9 тестов	5 положительных сценариев, пустой результат, неверные/повторные значения, диалог

Все 28 автоматических тестов завершились успешно. Онтология была пересоздана после исправления рецепта электронной схемы. Отдельно проверен запуск CLI для шести сценариев таблицы: ответы совпали с ожидаемым ранжированием и содержат объяснение. Внешний reasoner Protégé не запускался; заключение о полной OWL-консистентности поэтому не делается.

Команды воспроизведения

```
python3 src/lab1/ontology/build_ontology.py
swipl -q -s tests/test_lab1.pl -g 'run_tests,halt'
swipl -q -s tests/test_ontology.pl -g 'run_tests,halt'
python3 -m unittest discover -s tests -p test_dss.py
```

8. Оценка и интерпретация

В Prolog удобно записывать правила со связками и, или и не, а также рекурсивные зависимости материалов. OWL лучше фиксирует словарь классов, области свойств, диапазоны и ограничения кратности; онтологию можно расширять и изучать в Protégé. Для DSS выбран Prolog, поскольку выбор машины зависит от правил и численного ранжирования. Трассируемость поддерживает генератор: одинаковые имена продуктов, машин и ресурсов переходят из фактов в индивиды OWL.

Дальнейшее развитие: дополнить рецепты, разделить электрические и топливные затраты в единой метрике, добавить реальные параметры производительности и выполнять проверку HermiT в CI или Protégé. При расширении словаря важно оставлять проверки ссылочной целостности.

9. Заключение

Работают загрузка и запросы к базе знаний, генерация онтологии и диалоговая рекомендация по продукту, планете, энергии и приоритету. Ограничения результата — учебное подмножество Factorio и отсутствие проведенной проверки внешним OWL reasoner. Подход показывает, как прозрачные правила и явные ограничения помогают получать проверяемые объяснения и повторно использовать знания между логической базой, онтологией и приложением.